All right, good morning everyone. Do you see the zero two folder in challenges? You do see that, don't you? Yeah. That's your job today. So Martine's coming at eleven AM to speak, hopefully, for about an hour, so to lunchtime, I imagine. Um, and then we'll go for lunch. Um that's that. So uh you can get started from now till eleven. I will probably maybe I'll maybe interrupt you about ten AM or something to remind you to take a break. Um the DBT challenges, if you go through them, they should I mean different people take different lengths of time, that's why they're done as a self-paced thing. And they teach DBT as they go. So there's like reading and um and challenges to do.

Oh, it doesn't like sleeping forward. This is where I asked the helicopter pilot. Does flying other buildings cause turbulence or cause and he goes, Oh, I've never had that question before. Like, but you're a thirty thousand, apparently thirty thousand days of stunt pilot helicopter. How have you not had that question before? Yeah, that's vertical now. There you go. So there's those I think that's three hundred, those four hundred, is that right? So if you're at the top of uh at the top of the offices, we'll see the yellow the yellow yellow yellow helicopter. So I was in that hotel, the the the Bonaventure is it was called me. Um how much do you think that is that that helped a ride? Yeah, it was too eighty, I think. So if you want to do it. Compton, Compton helicopter. Uh I can try I mean I can maybe at some point. After break show you the show the beginning and end of the journey finding. Uh but um it was Compton helicopter airport school. If you want the actual um guys, um it's like celebrity celebrity helicopters. LA and that is the downtown LA one. And I thought it'd be nice to do the downtown LA one because of staying in downtown LA. Um there you go. You go over Dodger Stadium and things as well. There you go, there he is. Yeah. We fly we flew from Compton to downtown LA and then flew back. You can book one way things with helicopters, but they're more expensive than two. Yeah.

Alright. Where about in the gener phase one? Yeah. What's the interface one? Okay, right. Have you learned something? Yes. Good. I mean it's stressful as hell. Any of the creatures? Interesting question as to why does the state allow science

engineering? Why would it prepare powerful? I think we to eight, like in this. Yeah. Why would we put powerful science? Yeah. I think I might be. What was that? Listening time to you right. Got it. So the same thing as last time, but maybe you might want to alter the astronomer at the end. They don't want to invite astronomer back. Okay. We will do what we can. And would you mind this time data? I know we get a transcript last time that we need to time recording time. Just sort of put screenshots into the guide for people. I'm gonna leave this other time. You want to case with two time screen recording file new screen recording? Yeah, I'll snag it. Okay, okay. I don't know if it's time or project. Let's project it. Yeah. Yeah. Profly, roughly. Okay. So hey guys, my name is Martin. Um I know a lot of you, actually. Um I'm on platform, and so I'm here to talk a little bit about the platforms, how we might have come to the decisions we made on selecting the platforms. Um and then happy to answer any questions as we're going along, so you don't need to wait until the end to stop me. Oh, let me start the recording. Okay, so I mean, I'll start with airflow and then I'll move over to astronomer. Um ideas how many DAGs, Al, you're not allowed to answer. How many DAGs we've got in production? It's more than three. Yeah. No ideas? Alright, I will show you. So this is the open source as well. Uh managed by CG rather than we've got about 11,000, close to 12,000 DAGs live in production. And then about 15,000 total. A lot of those are actually off. Um so it's a pretty big airflow instance. In fact, when we chatted with astronomer, um, we asked them, like, hey, you know, we're thinking about moving to this federated world where everybody gets their airflow. What is like a size recommendation that you might have? And they're like, yeah, like at log, like the biggest one that you should have is about 2,000 dyes. So we're like way, way over that. Um test them into like ops and platform that we've been able to keep such a large airflow instance up and running for years at this point. Um, but also says a lot about where we need to go. So uh the way that today's Airflow instance works is it's like one massive EKS cluster. Um and then we've just got like this massive uh compute group where we just have like, I don't know, like you might know how like 900,000 CPUs on this cluster, and we just

scale up from zero up to like at some point we'll hit like close to that. I don't think we've ever actually hit the whole thing. Um so it's very large. Uh and then the other thing is we've got everybody within Caspian running on the Terraflow instance, which means we've got a lot of issues with noisy neighbors. So we'll have like a DAC potentially taking up uh resources that might be needed by another DAG, and so like one single deck has been a hundreds to even thousands of Spark executors. Um that becomes a problem, right? Because now you've taken resources away from other DAGs that you can. Um so that's that's a problem that we're dealing with today. Um we've been able to, you know, optimize some of these tags in some instances we need to increase the number of resources that we've that we've got. Sometimes we need to move jobs to different times just to like you know prevent some of that noisy name convention. Um and so we kind of realized we need to do something about it. Um so we have two options. We could have three options. We could create a bunch of EKS clusters and then put a bunch of airflows on top of these EPS clusters. Pro, we've done it before. Um, that's a lot of management to do for every single one of these EKS clusters. Um option two is going to AWS, M WAA. They have like a managed version of Airflow. Um it's easy to use. Uh we somewhat looked at it, but they didn't have really good scaling. Like we're dealing with the thousands of jobs, and at the time they only offered something called the seller executor. We use Kubernetes, and so we didn't really trust that celery could scale the way that we wanted it to. In addition to them not really having like a good PR solution, like a lot of our jobs, a lot of our data is like tier one data, right? Um it needs to be up. We have RPORTO requirements. They didn't really have that. Um and so then we looked at astronomer. Astronomer gave us uh the RPORTO promise at some point eventually. But with the promise that we will help you build a interim plugin that will give you those requirements, and it's 15, no more than 15 minutes data loss recovery within two hours. I think that the official CG1, I think, is five minutes of data loss, but like no SaaS can really offer that. So we're at 15 minutes. And so they helped us build a plugin. And since then they've actually released the

managed DR. So we're good there. They also have, you know, they're they're the largest contributor to the open source Airflow project. So like we're talking to the experts, right? They they've built Airflow from the ground up. And we get really good support. So we obviously decided to go with Astronomer. Oh, and the other benefit of Astronomer is they have a really nice CLI, which anybody can use, by the way. You don't have to be on Astronomer to use a CLI. They have a really nice CLI where you can run everything locally. And you can also really easily create deployments. So where we are today with Airflow on the PAPPAT for DXP, which is I think why you guys are all here. We have the ability to vend out an Airflow for every team pretty quickly through DevOps templates. It also comes with the repo, which I can look to next. The repo is pretty opinionated on the PAP path. It'll give you the ability to create DAGs. If you vent today, it might not require that, but hopefully in the next couple of weeks it will require it. And then it also is very opinionated on how on where these jobs run. So we're favoring data breaks, if you guys are probably aware. So that is Airflow. I glossed over a lot of things, but anybody have any questions on Airflow astronomer? I can dive into Astronomer as well if you need to. Yeah, just well, you noted today we're running not on Astronomer, right? We're running our own airflow. So what does the so there's target state, future state, uh what does the messy middle look like as we transition and move? Is that a hard cut or piece by piece? Yes. Good question. Um we are in the messy middle, so we've actually got production instances for Astronomer today. Um I can I can show you guys if you want to take a look. Um it's gonna be a transition, it's not gonna be a cutover. Um, and we're getting help from like within within Edo, we're getting help from the towers to help us move their pipelines, but also like outside of EDA, we're getting teams already onboarding and moving their pipelines. I will say a lot of what's in Astronomer today is manual pipelines outside of EDA. I think the Edo team is only moving, I don't know if you guys, I think you like CGU started moving OG started moving things. So it's gonna be a slow roll over months, maybe a year or two, hopefully not. But um, yeah, it can't be a cut over because you know, like there's so many

DAGs, and we need to move like the less critical ones over to Astronomer and then and then move on to the to the critical ones. You can't just cut over. Um, does that answer your question? Cool. Yeah, time well, timeline helps because I think in a lot of conversations, then we'll start with yeah, this is what we're doing now, but once we get to PAVPATH, then we'll have a chance to do things differently. So it's contextually, it's nice to know what part of PavePath is it? Airflow to Astro, or is it this to that, or this is the path, or this to that? Which one of those moving pieces we're talking about in that conversation? Yeah. When you guys move to the PAV path, we're gonna recommend starting with Astro. So like any net new jobs that you're creating should like we'll recommend you guys just go to Astro now for those net new ones and then start trying to move your old ones. Um does your team already have DAGs in open source as well. It's a little bit easier for you. You guys will just start creating these DAGs in Astronomer. Um you'll also get a data breaks that uh workspace that's specific for your team, um, where the Astro DAG is a trigger job onto. Okay. Um it's a little bit easier for you, but some of the teams that have DAGs already in Cascan, it'll take a little bit to transition. Here's a sample one. Um I guess the other thing is Airflow 2.11, which is what we're on in the open source world, is end of life. I think Airflow 2.11 to, yeah. So this is actually end of life. We've got extended support until April. So we need a we need to upgrade to Airflow 3 ASAP. Um so any new Airflow instance that gets vended through Astronomer is gonna be the latest version, 3.3. Um so it's gonna look a little different. If you're used to, if you're used to like the Airflow 2 view, when you come to Astronomer, you're gonna get something that looks like this. Um Astronomer gives you like the ability to change a lot of the UI, so like if you guys want to get fancy, you can make it look pretty custom, change colors at logos, whatever you want. Um, but the DAGs view looks a little bit different, so don't be alarmed, this is still Airflow. Um, it just looks a little different. Um you can see like this is still familiar, it looks somewhat the same. Um so yeah, this is this is an Airflow on Astronomer. Um this is no different than what Airflow 3 would look like in the open source world. The only reason the only real thing that changed is

the man the back end management of Airflow. A lot of the times, like you guys won't even need to get into the astronomer UI. You guys will probably just live in Airflow. Um because the astronomy UI is gonna be more like on the management of uh platforms and the operation side. Um so I don't actually expect many of you to need to go into the Astro UI. Um so yeah, this should be very familiar, maybe a little bit of a skin on top for Airflow 3. Um the other thing is we're gonna recommend going to something called the Astro Executor. Um it's just the way that jobs are managed within Airflow. So today, like I mentioned we use a Kubernetes detector. Um it's really good for scaling, spins up pods in Kubernetes, not to get too into the leads. Um, but every pod, if it lands on a new node, requires like an image pool that can take some time. Um is a little bit better for smaller deployments, but um the astro executor kind of bridges the gap and kind of brings to the best of both worlds. It's built on top of Celery, I believe, and they're rewriting it in Rust. Um like all new open source stuff is being where you're in in Rust. So hopefully it'll get us pretty good performance. But yeah, a lot of these are already on the Astro Insector. Um, I mean I can show you Astro too if you guys want. Um questions so far? I think I saw somebody over here. Yeah, which team has its own airflow. How do you use that? Oh, yeah, good question. Um it is it is in development. Uh we have a custom plug-in plugin called Scaffee, which sounds like a disease. Uh, but what it what it does, so Airflow has this concept of assets. And within an Airflow instance, you can publish an asset, and then any downstream DAG can just like subscribe to that asset. So you're no longer talking about like jobs, right? You're saying like I need my portfolio data, right? You subscribe to the portfolio asset. Uh, and then any downstream DAG that depends on on the writing of that portfolio asset is going to trigger on the completion of that. The problem is it works well inside of one Airflow, but when we bring in many Airflows by many for many different teams, you need to have like an API token that has access to that to that other astronomer, which we're not fans of here at CG. We try not to share these tokens that give like systems access to other systems. And so what we did is this plug-in that essentially lives inside of

your airflow that reads the asset that's being published, and then it pushes it to the downstream shutter instances. So to Airflow, it looks very much like it's all working in the same place because this plug-in is essentially like reading any asset from all of Capital Group and then publishing it to your other shot assets. So from your perspective as a data creator, you will you will just need to publish out a dataset like you normally would or an asset? The plugin will kind of handle that dependency for you. Similarly, when you create a downstream tag, it'll just subscribe to the asset. The plugin will handle like triggering for you. And we'll have samples in a causeway on how to do that. That plugin, is that using Kafka? Like as another reason? Yeah, so like to get in the weeds, the plugin is in magic. Like every one of these assets needs like some central control plane to manage everything. So when you publish an asset, the plugin is also pushing it to Kafka. Yeah, it's a new, it's a confluent one, so it's a new topic. But it's the same idea. So it'll it'll read the asset, push it to confluent Kafka, and then all of the other airflows via that plugin are consumers of that copy topic. So they'll read those notifications as they come in. Are you passing in the token, like the respective token or grab it? Yeah. Or yeah, this is getting like into the weeds. Um, but that's a good question. So we manage like the central platform AWS account, which has the confluent consumer and producer secrets. Um we expose those secrets to be public within the capital group or so that the plugin essentially hijacks a role inside of the AWS account and has access to Redet secret. So like you don't need to configure anything. When you install the plugin, the plugin knows the go read into our secrets manager and read that C group. This is just access Kafka then, right? Yeah, just for Kapka. Yeah. So we're no longer sharing like airflow tokens and stall Kafka. So is there like a central dashboard that you can check all of the published data? Uh yeah, I might get in trouble for sharing this. Um this isn't perfect, and we're still working on like the UI, because we're not a front-end team. Um, but the goal is that we will give teams the ability to see some sort of system lineage, uh, mainly with assets. I want to copy that this is not a lineage tool. Um so the intention isn't that we'll have like actual lineage here. I think we

still need to like talk about getting an actual linear system, um, data lineage system lineage, but like to some degree this is gonna help a little bit with like the discovery of assets. Um so you can see uh there's this job EOCG Sotera fact monthly. I don't know if you know, but it looks like it's a client group. Yeah, oh GoPal again. There you go, yeah. So um so you can kind of like take a look at some of these in here and drill down into like the details of it. Um so it looks like this one job is consuming all of these other upstream data sets. Um so again, it's something. Uh it'll help with discovery, but it's not really a lineage tool that we're gonna like. We're not saying this is the lineage tools rule at all. I think we still need an actual tool. Yeah, ops support things fail. You want to take that or you want me to take that? Oh man. So what we're what we're doing with with Al's team is uh we're working on like the platform side to like work together on figuring out what goes where um because we've also got an enablement team. Um from what we've been discussing, there needs to be an onboarding of ops for uh for your data creator team. So when you guys onboard the farmer creator DAG, when you guys are in the process of like, okay, we're testing things, we're gonna go to production. There needs to be an onboarding engagement with the operations team and a warranty period of some sort. I think that's to be determined based off of the complexity of the job. Um but you guys need to bring in the operations team, and by the way, the enablement team will help you with like that process. Um you'll bring in the ops team, show them the bags that you guys have on your environment, they'll ask for a criticality level uh questions. Um they'll ask for a warranty period because you know there's gonna be some sort of like KT between ops and your guys' data creator teams. Um and then at some point it'll put over to ops doing you know the level of support that was a good one. Um you'll get data. Auto vended for you, so you'll be able to see everything for your app Airflow instance. That happens today. In the future, we're gonna plug in the data bricks level metrics as well as that same dashboard. Um ops are gonna have is gonna have their own dashboards that kind of give like a broader view of all of airflow, um, and then they can drill down to your guys' stuff. Um astral. Cool. Well

is that? I would call it template points. No, um, autosys is still around. Um we are not involved with autosys at all, so I hope like it kind of depends on the teams that are migrating for the paid pathwork that involves data version airflow. So that's kind of been the standard in Caspian, so we're gonna continue. We really like Airflow. It's been it's been great. Like I said, we've got 14,000 jobs in there. Um the intention is not that this is going to replace OS's. Many teams are still in the Odysseys world and will continue to work there. Um we've got some bespoke linking between autosys and airflow via APIs. Um there could potentially be something we do there to improve that experience, um, but it's not really a priority of like the page at this point. Like any paper from all of the C installing. Yeah, you're asking what the integration pattern is for that. Yeah. Um it's API based if I'm not mistaken. So I'm not involved in that, um, but I'll give you what I believe is how this is working. Um there's an autosys box job that runs in the Alice world and then can trigger a downstream DAG based off of like a task at the end that just calls the Airflow API. I think that's the extent of the integration. I don't know if like some folks on the laser team might know better than I. There's an auto job where you can think it uses an API and triggers an airflow tag and you can just handle everything drawers instead. I think an integration the other way as well. There's an airplane task that can trigger an object down as well. Yeah. And it's already dead. Cool. So maybe I'll move on to data bricks. So I don't know. Have you guys done any data version? Well, I mean, whatever you want to say. Yeah, whatever you want to say. So databricks, we've been working on for like, I don't know, seven years or so. We started in the Azure world. We learned a lot in the Azure world. It was very basic when we started, it was just your compute engine. They have like a they had a local hive meta store where you can kind of like write your tables and create like external tables that are pointing really to like S3, but it had its limitations. You couldn't really share, and we're trying to move to this federated world of the PAVE path, and so like everybody's gonna get their own and have like a lot of teams already have their own database workspaces. But sharing data wouldn't have been great in the old hive metastore world. So

what Databricks did is they released a few years ago at this point now something called Unity Catalog, which is what we're standardizing on for the page path. With Unity Catalog, the data lives outside of your workspace. The data lives in this like magical world called Unity that finds the data into your workspace. And so there's gonna be a few types of, and then they have something called catalog. So this is a unity catalog. The Unity Catalog, like I said, is broad, it's not just for this workspace, it's just an example. Um the way the reason that these show up here and it looks like they belong to this workspace is because we bound these catalogs to this workspace. But really, like if I click on this, you can see workspaces. This one, this one is bound to these three workspaces. Um I could bind it to a different workspace if I wanted to. Here are all of our workspaces. Um just to give you an idea of like how the Unity catalog is set up. Um, so so the catalog, so the Unity catalog gives you all of these different catalogs that can be shared. Um inside of the catalog, you've got the schemas, and then inside of a schema, you have your data. Of course, I opened up one that doesn't have any data. Um so what what Capital Group has kind of aligned on are three main types of catalogs. Um application catalogs, domain catalogs, and source and a source aligned catalog. Application catalogs are bound only to one workspace, which is your workspace. So everybody's gonna have your application catalog. That's kind of your scratch pad, it's also a jump point between getting data from your application published to the domain catalog, which I can talk about. Um that's the application catalog, source line catalog. Um you can think of as like an ingestion workload if you guys are familiar with Cascade Ingestion. It is it's it's exactly what it sounds like. It's just uh data that is aligned to the source pretty cleanly, and there's not really a lot of curation. Like maybe there's a little bit of a translation in there. But the goal isn't that we're doing any real curation in the source line catalog. Um and then the domain aligned catalog is really where the curation steps are going to happen. Um there's two types of data in the domain catalog. There's gonna be um domain catalog data and then consumer aligned data, which is like maybe curated a little bit further so that it aligns better with like a

specific use case that can consume that data. Um so four real types of data, three catalog types. Um there's gonna be quite a few, there's gonna be many, many application catalogs because they're one to one with the workspace. There's gonna be a few domain catalogs, and then there's gonna be one source aligned out. So everybody's gonna publish the same source of line catalog. Um the access model uh is twofold. We started with we started with schema based access because in the Cascan world it's like five data type. So like if you want access to reference data, you request access to all the reference data, right? Um but we're hearing more and more use cases where like teams want specific object level access. Like I just want access to my table. I don't want to give others access to everything inside of the schema. Um so we now manually support uh table based access, object-based access. So if you guys have if you crack you guys create a data product and publish it to the domain catalog, um and you say like, hey, I only want this one table to be accessible via this one AB group. We can do that, it's a bit manual. We're working on automating. Um what we have automated today is schema level access. So that's kind of the standard that I started work by. Um let's see. You kind of just create additional schemas of Kanye if you wanted to read it. You can just create additional schemas yourself to refine the Kanye. So that's actually what we were recommending initially is like, hey, if you've got like multiple types of data that fall into one schema, why don't you just split that schema up into like two or three schemas, and then you keep just instead of two level at the schema? Um it works in some cases, but it seems like a little bit of an overhead to do like a single table in schema if we needed to. Um so it does work in some places, but we have the option to do table level on access as well. Yeah, and at some point we'll also have a bacteria for working on the patch of the based access. So you can have access to like only certain fields inside of the image. Sorry. So I'm gonna do the all case of the access. Yeah, you'll need like um, I think it's view on the schema and then like select on the table. But you won't be able to select on the whole schema. I I can show you um if it maybe if it clears it up, because I think it um this is probably useful. So you

can apply permission. I don't know what this is. This is the default schema. You can um maybe I can't apply permissions. Interesting. Oh, I didn't escalate, sorry, I should have escalated. Um, okay, so I could do like U schema, that's a LP schema. So I'll be able to see things in here, but I won't actually be able to select because we wouldn't use select on the schema. Um what we can do is at the table level, um, I can go in here and then give myself more whatever user. Um we do AD based access, we don't have individual access. So like OZ dev. Um I should be able to pull up one of these. I can get select access. So it is manual for now, but we've got like the architecture sign off that this is like the pattern for the end of the point. Any other questions? You need to cut a lot, is it? Yeah, or edition on the environment. Yeah, yeah, good question. It's per region. Um, yeah, exactly. So you you you technically could find this is why we don't really expose it. We have like a systematic way of finding. Um, but you technically could find a production catalog of production data in quit dev workspace um accidentally, and so yeah, it's at it's at the region level. Like all of Unity is all one with an RNL. So at the account at the region level. So it's it's kind of the same thing whether you create a new data level. Because essentially you're still creating the private. Yeah, for the management of like the access piece, yeah, it's the same idea. And so like it can be a very infinity when when someone uses table AL for this table, they don't they don't want to do this. So it can get PY That's a good segue into data contracts. So the data contract, um, we're gonna we follow ODCS standard and ODPS, so open data product standard. Inside of those YAML files, you'll have your access for and whether it's like schema level or um table level access. Um yeah, so like we're not trying to use like the the object itself as a discovery layer, we'll have the data contract and then marketplace as your discovery layer, which I guess also answers your question. Marketplace is also where you want to go to discover data. So the application catalog can argue shared across other teams, so it has to be mutated with consumer catalog, but maybe it is a temporary need, but they have to do it all site of data that I don't want to give an official temporary need access uh recommendation, because I'll shoot myself in the foot. Um I'll

give you the pattern. So the pattern is application catalogs can't be shared because we need to standardize on some of the requirements for all makes data, like domain data. Um, especially on like the operation side, we we need to make sure that your application is tier one, because like platforms are tier one, right? If you as a data creator are creating something, but your application is in tier one, and somebody else that is consuming that data is tier one, that becomes a problem during an outage. Um we also want to guarantee data quality rules. We want to do the best that we can with SLAs, although that's really out of our control. Um and so we we can't and access is the other thing, right? Like the whole point of unit catalog is that we unify access as well. Um if we start sharing data with like the spoke AD groups that people think is like adding cells to, we kind of lose the standardization on the access piece too. So the pattern is you write to an application catalog for you as a data trader or an application catalog, and then we will give you a blueprint that will copy that data from your application catalog after data quality rules run to the building catalog, and that's how you show data. Yeah. That's the reason why we have that application catalog. So the RLA can work right the application catalog, or uh, no, um, I don't think I don't think it will. Uh I don't think it can. We are trying to bring the two into one. Um, so like one, I don't know, but two, even if I gave you an answer, it could all change in the next couple of months. So we're working with the algorithm team, trying to like standardize on one approach for both the data path and for class. Oh question on the data tiering, right? So you noted the infrastructure and the platform itself is tier one, but then there's multiple layers of tiering that that platform itself supports. Um is the is the tiering then um adjudicated or decided on at an application level or at a different level, and then how is that maintained from an SLA SLL perspective platform? It's a good question. Um because unity catalog is for all of CG, we kind of have to standardize on like the Unity itself is tier one, whether your application is tier two or tier three. Um and uh and so like the DR conversation is probably one we'll need to have with your team because we're moving to manage DR, which means that you as a data creator are willing

to write DR rules for your data. Um so it it's it's kind of murky right now because we're we're going databricks just now released the MCR for like a few months ago, and like as we're testing these things, we're finding some like holes in the logic, and so it's in preview, it's not really GA at this point. Um so to be determined exactly on what that means, but um, yeah, we would essentially help your team with DR at this point and try to get you guys on like the tier one system. Um if you guys can't get onto tier one and like need to be tier two or tier uh tier three, you can have the talk and figure out how to do that. Um the source catalog and have to use catalogs. So I think the IMAP some data can use that, right? And they're expecting that to be in the source of the catalog. Right. Then in that application catalog are the public catalog? Ah, good question. Yeah, we need to figure that out because it is a question that keeps coming up. Like all data is curated to somebody at some point. Like it's source data to us, but like it's curated to IMAP, IMAP curated it, right? Um so we're architecture published some standards already that I can share with you. Um but because we actually haven't been using the source of the source aligned catalog uh as injunction at this point. I don't think that like the process yet uh has like a an opinion. Um but I will say like CETA, is it CDA? The there's a claim group team. C that yeah, okay. I think I think CEDA at this point is already publishing directly to the source line catalog that doesn't go through ingestion. Um I foresee that being a common pattern that continues to grow, where we'll give teams access to write to a source align catalog schema for their specific use case because there's no reason to like have that team publish a file out and then like us reconsume that file, spend compute just to get to the source line table when CEDA can publish or any other source line team can publish directly the source line catalog. Are they are they following the conventional standards? Like are they mimicing all the audit phones? No, they're not yet. There are standards that were like we're like in the process of building the paper path, so there's a lot that we need to get to. That is one of them. Yeah, we're alright. I mean, looking tool. I mean any other areas we want to talk about? Is that a tool or scheme label or fly label just paid? Uh yeah. Um we

have the flyaway capability now. So when uh part of like the platform vending includes a repo with like some templates, which include something we call data council, like the custom build tool that takes a data contract and converts it to DDL, and then the pipeline uses file to deploy that to data. Um yeah, uh I was gonna say one more thing. Could you go through is the EDL, the repository, get help repository, and just show people get a contract file and other. We have flyaway today. What we're finding is that well, at least personally, I think it's more overhead than it's like it's not really helping us too much. We already have source control. DDL is not the most human, I mean it's SQL is yet sure, like humans read it well, but like we have contract as well that have the same data. And you can always go back in time for that DDL. So what we will probably do, I think we still need to consult a little bit than ask people to help consult them with what I mean, is what I'm gonna say. We will probably favor the data contract as that DDL for flyaway, and then remove the flyaway piece uh in favor of just overwriting inside of your display pipeline based off of what's in a contract. Contracts have versions too, so like we can always go back to a previous version if we need to. Um so yes, to answer your question, we do have fly away. I'm not a fan of it, so we'll see if it sticks around or not. Um I don't think it's really helping in the catalog world. Um, I can move to the products repo. Um so this is very similar to the repo that you guys will get vended as data creators when we go through onboarding. Um stop me uh because there's a lot in here. There's a lot of like dot files. These dot files are just because we use like linters and pre-commit pre-push books. Um all the and obviously we've got like getting more get attributes. Um we use a tool called me. So like it looks a little messy, but it's mainly because we're using all of these tools to kind of like help with a local developer experience. Um so don't be alarmed, you probably will not need to touch many of these. Though you can if you want to, this is your repo, you can do whatever you want to do. Um the construct of it is um you can ignore these dots, like I mentioned folder. Um there's a data products folder, which is gonna include everything that you need to create a new data product on Astro DataBricks. Um there's the IAC pieces, which

include how to deploy your platforms. Um we've got some custom plugins, which really should be requirements text installs as Python packages instead of just like arbitrary phones filter. So that'll probably go away. Uh in fact it was gone. So this might be something custom that somebody's doing inside of this repo. Um they're shared assets, which can be invoked from your data product folders. Um obviously your tests, um, and then local wheels is a custom thing that we're doing with platforms. This isn't something that you guys have. Unless you want a team and create your own folder. Um so I'll mainly focus our time in the data product folder just because that's where the pipelines are going to be created. Um so the construct of the data products folder is again a couple more dot folders. We've got blueprints. The blueprints are some like pre-baked templates that we vent out into people's repos. Um they the templates are airflow templates. So like we stand we we created some standards uh in the open source world and we kind of migrated those into the astro world. Um some of those include like a dependency um up and downstream tags um using something we call tag exec check, um, which will go away, but for now, because we don't have like the scappy solution just yet. Um there's a blueprint for dag Zeccheck. There's a blueprint for how to run jobs on Databricks, so you don't have to reinvent the wheel and like write a custom task to go in your database. That's all kind of done for you. You just say use this blueprint and it'll kind of do it for you. Um and then you guys are welcome to create any blueprints that you guys want for any repeatable tasks within your teams. Um clusters, I'm gonna gloss over that because like really that'll probably move away into the app at this point uh at some point, but like this is just a way to create a Databricks cluster. Um if you wanted to have like a standard cluster that's always out there, um, you would do it here. Um pool, same kind of thing, uh your compute pools for the cluster. Um shared assets that relate to this data product, like we can think like models, like DVT models, anything that you guys want to share within this data product and go here. Um data foreman is something custom that we're doing inside of this data product repo. We we have it in some places in databricks, but I don't think

we can talk about that um here. Um we will so this is kind of where we start getting into the meeting. Um we will vent out some health checks by default into your guys' workspaces. Uh you can choose to keep those on or you can turn them off. They do cost money because this report, like it's like a 30-minute poll of like sort of health checks, right? Um these make more sense in the platform airflow instances. They probably don't make sense to run every airflow, um, but they're there if you want to keep them if you guys want to like make sure that you're able to hit all of your endpoints, you can add to them as well. Um so we'll bend those out. Um let me back up. There's another piece. We are we are hibernating all of your non-production airflow instances outside of like business hours. So if you guys need something custom, let us know. Like if you guys have an offshore team, we'll need to like make sure that the hibernation also accounts for offshore work. Um, but like you guys get charged at the deployment level at the airflow instance level. So having an Airflow instance up 24 by 7 is costing you money. Um so by default we do business hours, and then you guys can let us know if you guys want something custom. Um so that's also a reason why health checks may or may not be something that you want, because the health checks run 24 by seven. Um this is something we're working on, like I said, for the POC with uh VR. Um and then any any utils. Um Data Foreman is an actual data product. Yeah, okay. So moving on to an actual data product. Um inside of a data product, you can think of the whole thing. While you were there on like on the cost piece, um dashboards and cloudability regarding road slash who I forgot that was on type type of things that custom and data owners need to keep that in mind, or are we gonna get a check in once a month? Hey, do you guys have XYZ that's on you know kind of thing? Or a novel detection kind of thing? It'll be the latter. Like unfortunately for now we have to manually look at these things. Um Airflow just gave us a cost endpoint last month, and so we're gonna build like some sort of automation to potentially send you guys those reports without having without us needing to like view these things. Um cloud ability we're also trying to get. Um slammed um with working in the like around the AI space uh

two or four costs. So we don't think we'll be able to get cloudability anytime soon for both Airflow and Databricks. Um, but like we chatted with them one day, I think. Um so we're pushing as much as we can. Um it's unlikely that we'll be able to get that within the next seven quarters and this one. Okay. So once the closet out is that actually might not come on somebody else. I think we'll see. There's a couple of things that we have right now that might help. This is going to evolve. The data product owners are now owners of their own platform, right? Like we no longer have to put on like a central scene that fix the vulnerability at our speed, right? If you guys have, you guys did this, right? Or I think you were involved in it. If you guys have a vulnerability on a Thursday and you need to deploy it that day, there is very, very small chance that we'll be able to drop everything and then try to fix your vulnerability, right? So you guys own everything in here, pro and con, right? But yeah, script detection is going to be a problem. Problem in the sense that your stuff should still work. You just might not get the latest updates. There are two things. We've got uh monthly um release notes that are being sent out at this point through causeway by our product team. Um that'll help like keep you guys informed of things that we've released that might pertain to you guys. Um but that doesn't help you implement those things, right? That just alerts you that there are new things. Um so what we have in Catalyst is a skill that goes into our copier repo, which is where everything is sourced and updates your repo to the latest version. Um so you can just run that skill yourself when you see like a release that you guys like, um, and it'll just move all of those things in in for you, and then obviously you have to like approve a PR to make sure that it's not like changing something that you guys might have been custom because we expect teams to like customize a lot of things in here. Not all of them are gonna look the same. They'll be similar. Um, but yeah, we might change like a blueprint that you guys optimize for some like specific use case. Um so you'll need to like be aware of that, but hopefully that's the guys kind of uh be aligned with like it. Uh okay, so um at the data product level, uh this is a dab to Airflow, which is just a package, sorry, adapt to Databricks, which is just a package, I guess, deployed to

Databricks, which includes everything that you need. Um this is also a DBT project. Um inside of this DBT project, you'll have multiple data sets. Um I'll jump into here. Uh and the first one, oh, this is a bad example because this data set. Let me go to the ID project. We have we have some weird stuff that we want to play. So my asset reference data. Okay. So we've got two types of data sets in here, it looks like under the asset reference data, data product. Um link divisor. Um the idea with these repos is that everything that you guys need is inside of this data uh set folder. So you'll create your DBT, you'll create your Airflow jobs, your data, your um DAV, um YAML all within the same place, so you don't have to go jumping around multiple places. Um I'll start with Airflow. Um we are running low on time, so I will gloss over this. Um we standardized on something called astro blueprints. So you don't need to write like Python to create your DAG. You guys can just use YAML. Um the blueprints will read the YAML and turn it into a DAG and register it in. So you guys are abstract from all of that. Just write your YAML to create your job. Um the way that these are working right now is via the wires that I mentioned earlier. So we created a blueprint called DAG exec check. Um, and it takes these parameters. Um probably be common. So really the complexity is gonna lie in your pipeline or in your um in your SQL, not so much in the task uh in inside of Airflow, right? So we'll have these blueprints deployed. This is a task, this is it. This is how you create a task in Astro uh in the Airflow. Um, and you can see we should have a Databricks one data show now. So there's this database blueprint that depends on this upstream task that triggers this job in Databricks. So you can see it's pretty easy to create um DAC tasks in Astro on the repo because we kind of like deploy all of this for you. Um so that's Airflow. Um, and then just quickly glossing over your DAB, your bundle. So um we use apps for Databricks deployments. An Airflow job, all the all that airflow job is doing is it's triggering a job in Databricks. It's not actually running your SQL, it's just triggering a data breaks. Databricks is also running your SQL. To create that job, you need to you need to create a YAML for that job. This is where it gets a little bit more complicated. Um really the important part here is like these lines

here. This these are the actual commands that your data's job is running. Um, and you can see it's pointing to the files that you might need and it's referencing any like variable that you might want as part of this DVT job. But this is really you guys are gonna go through like how to run DT if you haven't already. Um but this is this is how you you could uh construct your your dab uh on on database. Um then maybe I'll end the contracts. So like I mentioned, we've got OECS, OPS. Um, you publish to a domain catalog, these are requirements. We need to have an open data contract file. Um so that we standardize on like what a data domain um table looks like. Um and these can be shared. Um we're gonna seed these into marketplace and then like grab all of the relevant information so that our registered marketplace, and then anybody can discover these data sets. Um we don't yet have this, but we will also pretty soon deploy a CICD time check to make sure that the contract both exists and aligns with the standards that way anybody that's consuming can read the contract to stand up. Um so there'll be some requirements, and and I can share. Maybe I'll Michael share the uh the confluence page with the with the contract spec if you want to share it with the everybody. Um yeah, open data contract standard is what we're using. Uh it's YAML and it should have the schema. Um this will evolve when we get to ABAC, which when we do attribute level access, we'll probably need to have some sort of like identifier in here to develop whether like a group does or doesn't have access at the attribute button. Cool. Any questions on that? At the bottom of compute? Yeah. Serverless is gonna be preferred. Um like I almost think that we will have very, very little classic compute at this point. Um when you mend out your Databricks instance, it's gonna be vended out on top of your AWS uh account, and that'll spin up actual PC2 instances in the background when you use classic compute, um, which require like warm-up time because you're using pool right and pull cost money. Um serverless is what we prefer, it's gonna be quicker. And so like we'll try to vent out of the data.config with your with your uh repo and your data grace workspace so that like when you create a serverless job, you also see the metrics the same way you would see like your classic plus.

If you have something that's super sensitive, you might want to use class on view. Yeah. We've got approval to use serverless. Um we also have white listed a bunch of uh endpoints that can come back and talk to CG. So like the most common ones should already be white. But if you have something that requires you reaching back into CG that isn't hard listed, the name of the team should be able to help you have this stuff. Cool. Cool. That is it. Any questions before we're up? For any additional questions I can send you some. Yeah. Do you have this on the book spaces on the uh we've got an Excel spreadsheet somewhere? Um get fast and user sometime and micro. Yeah. There's a consumer workspace concept that we're working on where like business users that don't really align with integrator team can jump into these workspaces that are shared. Um that's probably what we'll align. Like we it doesn't make sense to give uh like business users access to the creator workspaces at this point because you can have more data than really need. Um pipeline generation and publishing under the main catalogs not for users. So isn't that just isn't that just hard replaced or not? Uh what part? The discovery of the workspaces or no. No, not that a workspace level. Mark is helping you with the data. But it won't tell you the platform information. Gotcha. Yeah, so um a better answer is maybe when we come up with like the the consumer workspace um concept, that we should share. And there might be like a there might be like a domain line consumer workspace for that domain data that that uh business users can just jump into and consumer data. Think like Dremio like where you just kind of jump in there and the information that you need. Um but that's uh an architecture that we're still working on. Uh if you want like on site speed and phase though, like the old fan like a I'm a fan of them. Um I'm thinking that like maybe the source align things could standardize on like label jobs, like full pipelines. Um they're not I'm gonna send I'll send you guys another like approved features item because I don't remember if that's actually something that we'd like to approve. We definitely haven't standardized on it. Like we don't have standards on how to figure pipelines. If it's approved, it's something that you might be able to just do for your

team. Um, but as part of the paved path, it's not something that would be built in. So you have to like figure out your CIC component for those pipelines yourself. But I'll share, I'll I'll share the I'll share a couple documents that um that we can take a look at from. Yeah, good question. I'm a fan. I I think they've really cool to use the create website. Alright. Stop doing the recording. Cool. And if you could share that in your team by teams or something, yeah, I'll bring it to you if you want to share it. Okay. Cool. Uh should you come back after lunch? All right, okay. Thank you. Um Martin, can I grab you for a second? Sure. Uh just that repo, yeah. It still comes up. Um logged in as near. I don't know. I don't run the GitHub team, so I don't I don't know how they manage like getting your identity created. So Michael tried to add me, but yeah, no, I didn't come up. Yeah, I think I think you need to be registered through GitHub. Can you get a BSP? Can you ping BSP? His initials. BSP. Yeah, he might be able to guide you in the right direction on on who to reach out to. Yeah. Um I ping him directly. He might not like ping directly because I think they have like groups and stuff. But um yeah, just ping them and see if we can give you a uh contact for how to get access. But yeah, I didn't see you in there. I I tried manually adding it too and it wasn't you in there. All right, so thanks anyway. I'll reach out and see how Michael, do you want to go grab your lunch? Um we have got a call with Jasper.

So GBT pipelines themselves can carry out the full good. What else? Interesting? Just in the schema? Hey? Catalog. Catalog. Yeah. Uh what else? Does anyone know about uh each? We've got some people about know they know about bi-temporal loading. Yeah, well, I know you I know you know. Do you know about the bi-temporal loading? Why don't we have tomorrow morning? The people who know about bite. Tell us about bite temporal loading. What the columns are and one or two strategies for bite number loading. Yeah. Okay, good. There we are. We've got that. Um we have some uh what about um have access to harness and um CICD stuff? Do? Oh I guess. Yeah, there we go. We've got we've got a nice Thursday morning lining up, right? So there we go, put that lined up. Now, uh of course the background processes there are not always directly relevant to the

narrative of data creators because it's in context. Uh you can see it over it, but that's good to do something in the background. What else are we going to learn about DBT? Good! That's a lot. Uh we've learned we can generate the catalyst. Uh we can learn that we learn that 80 to 9% of our job depending on the complexity of the thing could be generated. Good. Uh we learn that we put it in a repo provided by EDO at the own. Uh called Edo trained data products, or not training, sorry, Edo data products for the real one. Yeah. We learned that there's a data products folder within the repo where our code goes. So when we're on the capsule event on Friday, and we're running code in the wrong folder, that will be on us, won't it? It will take collective as well as individual responsibility for running the thing in the data product floor. Cool? Or try to, finally. But now you've been told that. What else have we learned? We've learned that in uh we've learned this is something we've learned for tomorrow, I guess, but we've learned that in Airflow there are these things called blueprints that you can fill out with YAML or tomorrow. How does data rich run DBT? Does D does David run DBT code directly? Does it sort of interpret DVD code? It does. Uh, what is the one type of code that David does run? Yeah, five letters. I knew there was something wrong with that. I was like, no, it doesn't look great. Five letters. Meaning with S. So it only runs Spark. So all code that Debix runs is actually translated into Spark. You know about Spark, you know about RDDs, we delete the screen of data sets, all that sort of stuff, so we can fringe things. So you don't have Spark works if it's just trusted to Spark. Um where does data that lives in DB uh so the TBT is running on Databricks and it's running on files in S3? And on the paved path by default, um those files should be in what format. Delta format. If they're not in delta format and don't have acid, you don't have acid linearizability, I don't need linearizability, you don't have consistency guarantees in the time. Cool. So what the delta format, I know that was a fast than one thing. But what that uh what Delta format does is it's a columnar format, meaning it's a file format where you store data in column-wise rather than row wise. So exactly like CSV is stored row-wise. You scroll, and you scroll rows. You know, unless you've got an ID

column, you don't need all the IDs will be listed first. Well you then uh and then uh then you have uh Alice and you have in all your in all your names, then everything else. Except you scroll in the file, get one column and then we'll follow the column. The reason for that is of course is that in analysis, you typically want to select columns, don't you? And you may be filtering by columns more than you're filtering by rows. And typically speaking, when you derive tables, it will filter the rows away, so that the next step is taking all the rows from there. Do you think it's called columnar column R formats? Delta is a column R whole narrative format with uh internal logs and manifests and logs that allows the system to have massive guarantees. Cool. Now if you worked with data X N S3 before, um parquet files for example, you may be aware that it's very hard to insert and delete rows and other things in those files that would not apply to delta files. Maybe hard, I don't know, but you can certainly insert data into data files and whether we can't do it, for example, parking files. How many different types of unity catalog are there on the CG policy? How many are there supposed to be? What the founders? Three. Ah, domain and source, right? I guess. That's great. I agree, that is great. Um give me one more factor we've learned and then I think magically because the clock might turn turn through. 40 minutes into the future. One more thing. Can we have one more surprising thing we've learned? I think it's pretty easy to have data quality tests throughout your files. Yeah. It is, yeah. How do we consider magic? Where do we consider materialization? We can configure it into two places. Where are those two places? And the actual model file itself, we put it in the braces and say materialization. On that bomb shot. And that's surprising, amazing fact. We can all go away. Someone call the president, you can change materialization within the model file itself. Of course he doesn't know, he's asleep on dialogue.